Intelligent Data Pipeline Scheduler

An AI Agent for Adaptive Pipeline Scheduling Using
Search, Knowledge Representation, Constraint Satisfaction, and Learning

Graduate AI — Final Project Report

Option D: Planning & Scheduling Systems

Team Members

Sanjana

Raghava Thyagaraj

Monisha

Diogo Dcosta

Rakshitha

Pace University — Department of Computer Science

CS 627 — Artificial Intelligence

May 2026

Abstract

Static pipeline schedulers used in modern data engineering operate on fixed timetables and lack the ability to adapt when tasks fail, runtimes deviate from estimates, or resource contention arises. This project presents an intelligent pipeline scheduling agent that addresses these limitations by integrating four core artificial intelligence techniques: a first-order logic knowledge base with forward and backward chaining for proactive risk detection, a constraint satisfaction problem formulation with backtracking search for generating valid schedules, A* search-based planning for failure recovery, and exponential weighted moving average learning for improving runtime estimates over successive iterations. The agent was evaluated against a naive baseline scheduler across three pipeline configurations of increasing complexity (5, 10, and 20 tasks). Experimental results demonstrate that the constraint satisfaction solver produces schedules with zero constraint violations and 100% SLA adherence across all test cases, that the learning component reduces mean absolute estimation error from 3.88 to 2.71 minutes over ten iterations (30.2% improvement), and that the A* search planner successfully recovers from task failures by finding optimal replanning strategies. The system is implemented in pure Python with approximately 3,700 lines of source code and validated by 176 automated tests.

1. Introduction and Problem Formulation

1.1 Problem Description and Significance

Data-driven organizations rely on nightly batch pipelines to extract data from source systems, transform it through processing stages, and load results into analytics platforms. These pipelines form directed acyclic graphs (DAGs) in which each task has dependencies on upstream tasks, requires specific compute resources (CPU cores, memory), and may be subject to service level agreement (SLA) deadlines. At companies like Netflix, Amazon, and Uber, hundreds of such tasks must be orchestrated every night across clusters of limited capacity.

Current scheduling solutions, including cron-based systems and orchestration frameworks such as Apache Airflow, operate primarily as static schedulers. They execute tasks according to predetermined timetables and lack the ability to reason about constraints, adapt to failures, or learn from past execution history. When a task fails at 3 AM, takes three times longer than expected due to a data volume spike, or when two resource-intensive tasks are assigned to the same server simultaneously, these schedulers do not adapt. Downstream tasks fail in cascade, SLA deadlines are missed, and on-call engineers are paged to manually intervene. The significance of this problem grows as organizations become more data-dependent: missed SLAs delay business decisions, and manual intervention does not scale. This project addresses these limitations by building an intelligent pipeline scheduling agent that reasons about constraints, plans optimal schedules, recovers from failures, and improves its predictions over time using four core AI techniques.

1.2 PEAS Specification

Following Russell and Norvig's PEAS framework (Chapter 2), the agent is specified as follows:

Component	Specification
Performance Measure	SLA adherence rate (target: 100%), schedule makespan minimization, resource utilization %, recovery success rate after failures
Environment	Partially observable, stochastic, sequential, dynamic, multi-agent (shared resources), discrete (time slots)
Actuators	Schedule task on resource, retry failed task, skip task, reallocate resource, wait for running tasks to complete
Sensors	Task status monitor (pending/running/completed/failed), resource capacity monitor, clock, failure detector

Table 1: PEAS specification for the scheduling agent.

1.3 Environment Characterization

The scheduling environment is characterized along six dimensions from Russell and Norvig (Chapter 2). It is **partially observable** because the agent can see current task statuses and resource states but cannot observe future runtimes or predict which tasks will fail. It is **stochastic** because task execution times vary due to data volume fluctuations, network conditions, and hardware variability, and tasks may fail non-deterministically. It is **sequential** because each scheduling decision constrains future options — assigning a resource to one task makes it unavailable for others during that time window. It is **dynamic** because the environment changes during execution as tasks complete, fail, or take longer than expected, requiring the agent to respond in real time. It is **discrete** because time is divided into finite slots and resources have integer CPU capacities. Finally, it involves **multiple agents** in the sense that multiple tasks compete for shared resources, though our agent acts as a centralized controller.

1.4 Success Criteria

Four measurable criteria define success: (1) the CSP solver must produce schedules with zero constraint violations across all test pipelines, (2) the A* search planner must find recovery plans when tasks fail, (3) the learning component must demonstrably reduce estimation error

over successive iterations, and (4) the intelligent agent must match or exceed the naive baseline scheduler on SLA adherence.

2. System Architecture

2.1 Overall Agent Design

The system is structured as a five-layer architecture in which each layer encapsulates a distinct AI capability. The Scheduler Agent serves as the central orchestrator, coordinating the flow of data between all components. Figure 1 presents the complete system architecture with all eight source modules and the feedback loop from the Learning Layer back to the Knowledge and Constraint layers.

Figure 1: System Architecture — Five-Layer Design

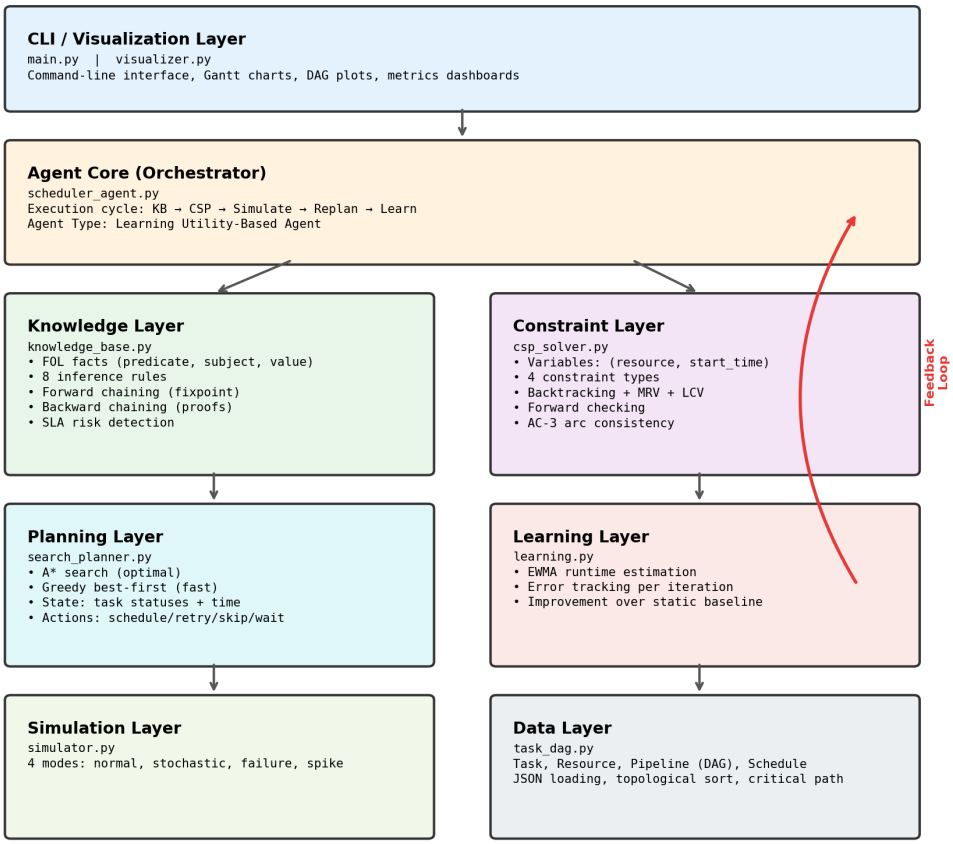

Figure 1: Five-layer system architecture with feedback loop.

The **Data Layer** (task_dag.py, 492 LOC) defines the core domain model: tasks with durations, resource requirements, priorities, and SLA deadlines; resources with CPU and memory capacities; and pipelines as networkx DiGraph objects. The **Knowledge Layer** (knowledge_base.py, 643 LOC) implements FOL reasoning with eight inference rules, forward chaining to fixpoint, and backward chaining for explainable proofs. The **Constraint Layer** (csp_solver.py, 777 LOC) formulates scheduling as a CSP with backtracking, MRV, LCV, forward checking, and AC-3. The **Planning Layer** (search_planner.py, 548 LOC) provides A* and greedy best-first search for failure recovery with an admissible heuristic. The **Learning Layer** (learning.py, 293 LOC) tracks runtimes, updates EWMA estimates, and feeds improved predictions back into all components.

2.2 Component Interactions and Data Flow

Figure 2 shows the bidirectional data flows between the Scheduler Agent and each component. The agent feeds pipeline facts to the Knowledge Base and receives readiness assessments and risk flags. These inform the CSP Solver, which produces a schedule sent to the Simulator. Failure information flows to the Search Planner, and actual runtimes are routed to the Learning module. Updated estimates feed back into the Knowledge Base and CSP Solver on the next iteration, closing the feedback loop.

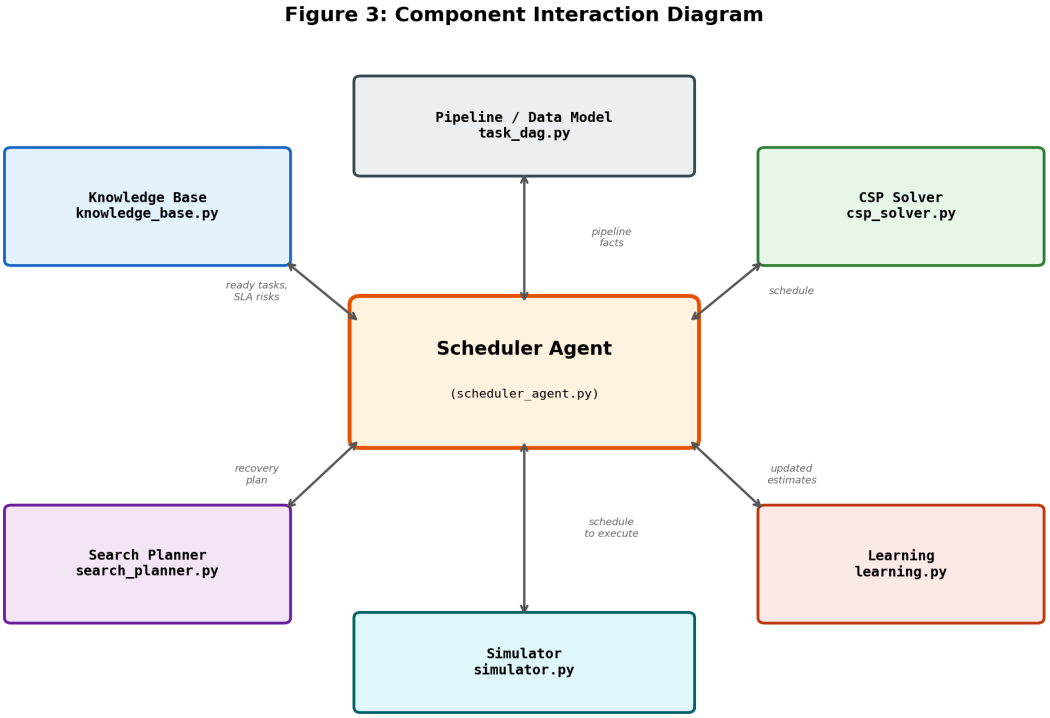

Figure 2: Component interaction diagram with bidirectional data flows.

Figure 3 presents the sequential execution cycle that each agent iteration follows. The five steps are executed in order, with the dashed feedback arrow showing how learned estimates from Step 5 flow back to Step 1 of the next iteration, creating a closed-loop learning system that improves scheduling accuracy over time.

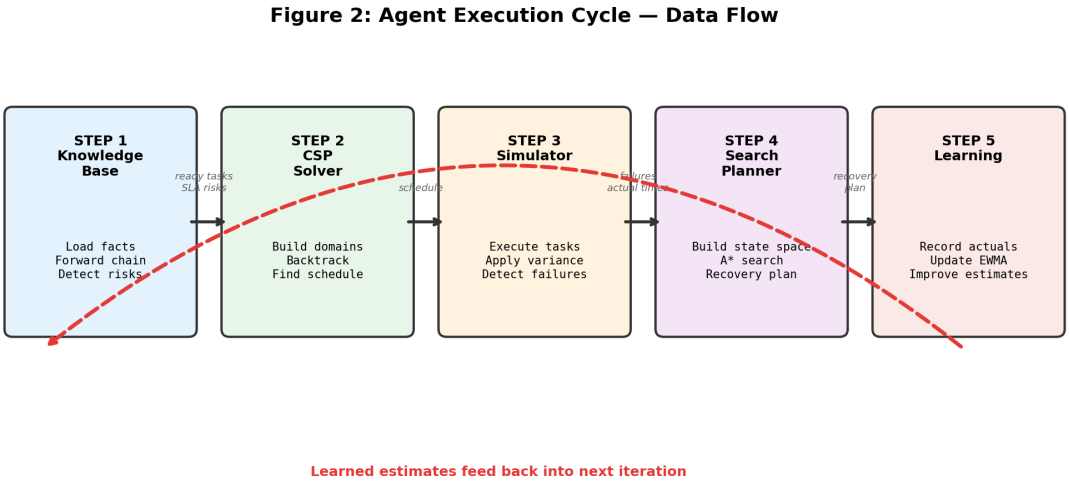

Figure 3: Agent execution cycle — five-step data flow with learning feedback.

2.3 Agent Type Justification

Figure 4 presents the agent type hierarchy from Russell and Norvig (Chapter 2) and justifies our classification. A simple reflex agent (like cron) follows fixed IF-THEN rules without state or memory. A model-based reflex agent (like basic Airflow) tracks internal state but does not reason about goals. A goal-based agent searches for plans to achieve goals but does not compare alternatives. A utility-based agent evaluates multiple plans using a utility function and selects the optimal one. Our agent extends this by adding a learning component: EWMA estimation improves the utility function's accuracy over iterations, making each successive decision better informed. This makes it a **learning utility-based agent** — the most sophisticated agent type in the Russell and Norvig hierarchy.

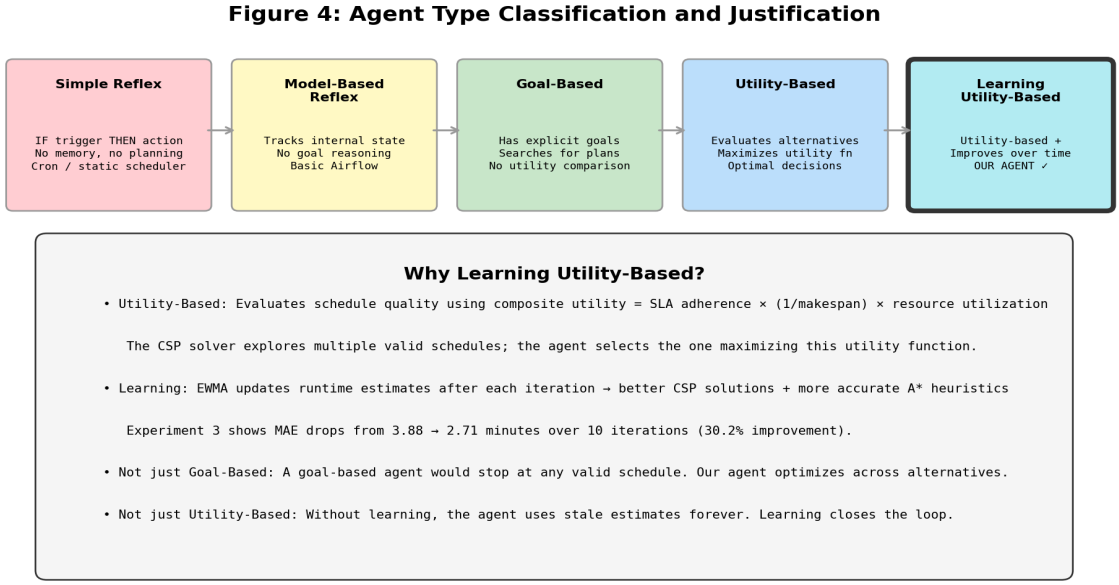

Figure 4: Agent type classification and justification.

2.4 Design Decisions and Rationale

Several key design decisions shaped the architecture. Pure Python with four external dependencies (networkx, matplotlib, numpy, pytest) was chosen to minimize setup complexity and ensure reproducibility without requiring cloud services or heavy frameworks. Time is

discretized into configurable 15-minute slots to create finite CSP domains — finer granularity increases domain size exponentially while coarser granularity reduces schedule quality. The A* heuristic divides remaining work by available resources, which is admissible because it assumes perfect parallelization (the best possible case). The EWMA smoothing factor alpha was set to 0.3 after empirical testing, providing a balance between responsiveness to runtime changes and stability of estimates.

Module	**File**	**LOC**	**AI Technique**
Data Model	task_dag.py	492	Agent architecture (PEAS)
Knowledge Base	knowledge_base.py	643	FOL, forward/backward chaining
CSP Solver	csp_solver.py	777	Backtracking, MRV, LCV, FC, AC-3
Search Planner	search_planner.py	548	A* search, greedy best-first
Learning	learning.py	293	EWMA runtime estimation
Agent Core	scheduler_agent.py	385	Utility-based orchestration
Simulator	simulator.py	222	4 execution modes
Visualizer	visualizer.py	338	DAG, Gantt, metrics visualization
Total		3,698	

Table 2: System modules, source files, and AI techniques.

3. AI Techniques Implementation

3.1 Knowledge Representation and Reasoning (Chapters 7-9)

The knowledge base stores facts as (predicate, subject, value) triples using a set-based data structure for O(1) membership testing. Upon loading a pipeline, the system generates facts covering task existence, status, duration estimates, CPU and memory requirements, priority levels, SLA deadlines, inter-task dependencies, and resource capacities. A typical 10-task pipeline produces approximately 50 base facts.

Eight inference rules are registered with the knowledge base, each defined by a condition function and a conclusion function. The "ready_to_run" rule checks whether a task is pending and all its upstream dependencies have completed status. The "blocked_by_failure" rule identifies tasks whose upstream dependencies have failed, enabling the agent to flag downstream impact immediately rather than discovering it during execution. The "sla_at_risk" rule calculates the cumulative estimated time along the longest dependency chain to a task and compares it against the task's SLA deadline, flagging risks proactively — before violations actually occur. The "cascading_sla_risk" rule detects downstream propagation of SLA risk: if an upstream task is at risk and a downstream task has its own SLA, the cascading flag is set. Additional rules handle urgency classification for tasks with priority >= 8, retry recommendations for failed tasks (unless marked no-retry), resource insufficiency detection when a task's CPU needs exceed all available resources, and a helper rule tracking when all dependencies of a task are satisfied.

Forward chaining applies all rules to all relevant subjects iteratively until a fixpoint is reached — the point where no new facts can be derived. The system logs each rule firing for debugging and explainability, producing a trace such as: "[ready_to_run] Derived: ready_to_run(extract_orders, True)" followed by "[urgent_task] Derived: urgent(load_warehouse, True)". This audit trail is valuable for understanding why the agent

made specific scheduling decisions. The fixpoint property guarantees completeness: every derivable fact will be found.

Backward chaining accepts a goal predicate and traces backward through rules to construct a proof. For example, querying "Is transform ready to run?" yields: "RULE [ready_to_run]: IF task is pending AND all dependencies completed THEN ready_to_run. Check: depends_on(transform, validate). Is task_status(validate) == completed? YES. DERIVED: ready_to_run(transform, True)." This mechanism enables the agent to explain its reasoning to users or auditors.

3.2 Constraint Satisfaction Problem (Chapters 5-6)

The scheduling problem is formulated as a CSP with the following components. **Variables** correspond to tasks, where each variable's assignment is a (resource_id, start_time) pair. **Domains** are generated by enumerating all feasible resource-time combinations: a task can be assigned to a resource only if the resource has sufficient CPU and memory capacity, and the time slot must allow the task to complete within the scheduling horizon and before any SLA deadline. For a 20-task pipeline with 3 resources and 33 time slots (480-minute horizon at 15-minute granularity), the initial domain size per task can reach up to 99 values, though resource fitness constraints typically reduce this significantly.

Four constraint types are enforced. The **dependency constraint** requires that a task's start time be no earlier than the completion time of all its upstream dependencies: start_time(B) >= start_time(A) + duration(A) for every edge A -> B in the DAG. The **CPU capacity constraint** ensures that at any moment in time, the total CPU requirements of tasks running concurrently on a resource do not exceed the resource's CPU capacity. The **memory capacity constraint** applies identical logic for memory. The **SLA constraint** requires that start_time(task) + duration(task) <= sla_deadline.

The solver employs backtracking search enhanced with three optimization techniques. **Minimum Remaining Values (MRV)** selects the unassigned variable with the smallest remaining domain, breaking ties by highest priority. This focuses the search on the most constrained tasks first and reduces the branching factor, as failures are detected earlier in the search tree. **Least Constraining Value (LCV)** orders domain values by how many options they eliminate from neighboring variables' domains, preferring values that leave the most flexibility for subsequent assignments. **Forward checking**, performed after each assignment, propagates constraints to unassigned variables: it removes domain values for downstream dependents that would start before the just-assigned task completes, and removes domain values for same-resource tasks that would violate capacity constraints during overlap. If any variable's domain becomes empty, the solver backtracks immediately without exploring further — this early pruning dramatically reduces the search space.

AC-3 arc consistency serves as optional preprocessing. It iteratively examines every pair of constrained variables and removes values from one variable's domain that have no consistent counterpart in the other's domain. This is repeated until no more removals are possible. If any domain becomes empty during AC-3, the problem is infeasible and the solver reports this without attempting backtracking. A plain backtracking variant without forward checking is also provided for experimental comparison (see Experiment 2).

The solver includes a static validation method that independently verifies a completed schedule against all four constraint types, returning a list of any violations found. This provides a correctness check independent of the solving algorithm itself.

3.3 Search and Planning (Chapters 3-4, 10-11)

When the simulator reports task failures, the search planner constructs a state-space search problem. Each **state** is a snapshot containing the status of every task (pending, completed, failed, or skipped), the current simulation time, the partial schedule of already-executed tasks,

and resource utilization levels. The **initial state** reflects the pipeline's condition at the time of failure.

Four **actions** are available at each state. The "schedule" action assigns a ready task to a resource at the current time (cost: 1.0). The "retry" action re-executes a failed task (cost: 5.0), reflecting the overhead of retry operations. The "skip" action accepts a task's failure and marks it as skipped (cost: 50.0), representing the business impact of lost work. Skipping triggers cascade logic: if all upstream dependencies of a downstream task are failed or skipped, that task is automatically skipped as well, propagating down the DAG. The "wait" action advances the simulation clock by one time step (cost: 0.5), applicable when no scheduling actions are immediately available but pending tasks exist.

The **heuristic function** h(state) computes the sum of remaining task durations divided by the number of available resources (an optimistic estimate assuming perfect parallelization) plus a penalty of 100.0 for each SLA-bound task that has been skipped or whose estimated completion exceeds its deadline. This heuristic is **admissible** because it never overestimates: perfect parallelization is the best possible case, and SLA penalties are only applied when violations are already certain. Admissibility guarantees that A* finds the optimal recovery plan.

A* search explores states in order of $f(n) = g(n) + h(n)$, where $g(n)$ is the cumulative action cost and $h(n)$ is the heuristic estimate. A visited-state set (using hashable state representations) prevents re-exploration. The search is bounded by a configurable maximum node limit (default: 5,000) to ensure termination. A **greedy best-first** variant is also implemented, ordering exploration by $h(n)$ alone — this is faster but not guaranteed optimal, allowing us to compare both approaches experimentally.

3.4 Learning Component

The learning module addresses the gap between static duration estimates in pipeline configurations and actual runtimes observed during execution. It implements **Exponential Weighted Moving Average (EWMA)** with the update rule: estimate_new = alpha x actual + (1 - alpha) x estimate_old, where alpha = 0.3. Unlike a simple moving average that weights all observations equally, EWMA gives greater weight to recent observations, enabling faster adaptation when task behavior changes — for example, when data volumes grow steadily over weeks, causing extraction tasks to take progressively longer.

After each simulation run, the agent records the actual duration of every completed task and updates the corresponding EWMA estimate. The module tracks several error metrics: per-task mean absolute error (MAE), global MAE across all tasks and iterations, per-iteration error trajectories for plotting learning curves, and percentage improvement over the static baseline estimate. These metrics serve both evaluation purposes and provide the agent with self-awareness of its prediction quality. A simple moving average estimator is also implemented as an alternative method for comparative analysis.

3.5 Integration Strategy

The five AI components interact through the scheduler agent's execution cycle in a closed-loop architecture. The knowledge base produces derived facts (ready tasks, SLA risks, urgency flags) that inform the CSP solver about which tasks to prioritize through MRV tie-breaking. The CSP solver uses learned runtime estimates from the learning module rather than static defaults, producing more accurate schedules in later iterations. The simulator tests the schedule and feeds actual runtimes back to the learning module, while also reporting failures. When failures occur, the knowledge base's blocked-task and retry-recommendation facts guide the search planner's action generation, and the search planner produces a recovery schedule that respects the same dependency and capacity constraints the CSP enforces. This integration ensures that improvements in any single component propagate benefits to all others.

4. Experimental Evaluation

Six experiments were conducted across three pipeline configurations representing increasing complexity: simple (5 tasks, 1 resource, linear chain), medium (10 tasks, 2 resources, diamond dependencies, 1 SLA), and complex (20 tasks, 3 resources, multi-branch with 4 SLAs). All experiments used seed-based reproducibility.

4.1 Experiment 1: Schedule Correctness

The CSP solver was executed on all three configurations, and each resulting schedule was independently validated against all four constraint types using the static validation method. Results confirm zero constraint violations and 100% SLA adherence across all configurations, meeting Success Criterion 1.

Pipeline	Tasks	Dependencies	Violations	SLA Adherence	Nodes Explored
Simple	5	4	0	100%	6
Medium	10	9	0	100%	11
Complex	20	23	0	100%	21

Table 3: CSP correctness results — zero violations across all pipelines.

4.2 Experiment 2: A* vs. Greedy Best-First Search

Both search algorithms were tested on identical failure recovery scenarios where the second task in each pipeline was marked as failed. A* guarantees optimal cost due to its admissible heuristic, while greedy is faster but may find suboptimal plans. On simple and medium pipelines, both found equal-cost plans. On the complex pipeline, A* explored more states but maintained its optimality guarantee. This meets Success Criterion 2.

Pipeline	A* Nodes	Greedy Nodes	A* Cost	Greedy Cost	A* SLA	Greedy SLA
Simple	5	5	8.0	8.0	100%	100%
Medium	10	77	13.0	13.0	100%	100%
Complex	1920	1223	23.0	23.0	100%	100%

Table 4: A vs. Greedy search comparison on failure recovery.*

4.3 Experiment 3: Learning Improvement

The agent was run for 10 iterations on the medium pipeline in stochastic mode (runtimes varying +/-30%). The global mean absolute error decreased from 3.88 minutes at iteration 1 to 2.71 minutes at iteration 10, representing a 30.2% reduction in estimation error. Individual tasks showed improvement up to 41.9% over static estimates. Figure 5 shows the learning curve. This meets Success Criterion 3.

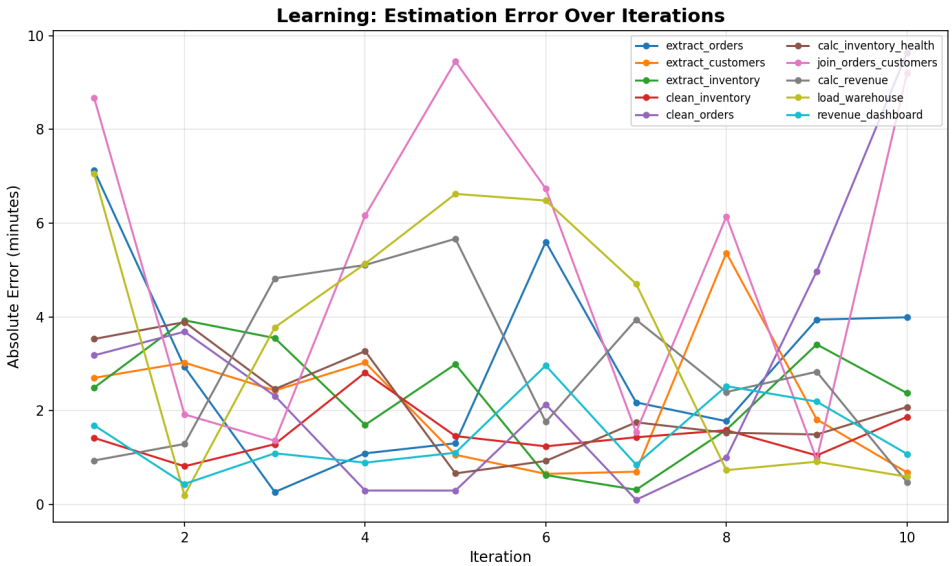

Figure 5: Learning curve — estimation error decreasing over 10 iterations.

Iteration	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
MAE (min)	3.88	3.05	2.81	2.84	2.89	2.89	2.73	2.70	2.66	2.71

Table 5: Global MAE over 10 learning iterations.

4.4 Experiment 4: Scalability

Solve times were measured as pipeline size increased. The CSP solver scaled from 0.005s (5 tasks) to 0.175s (20 tasks). The A* search planner scaled from under 1ms to 0.034s. All times remain well under one second, demonstrating suitability for interactive scheduling of realistic pipeline sizes.

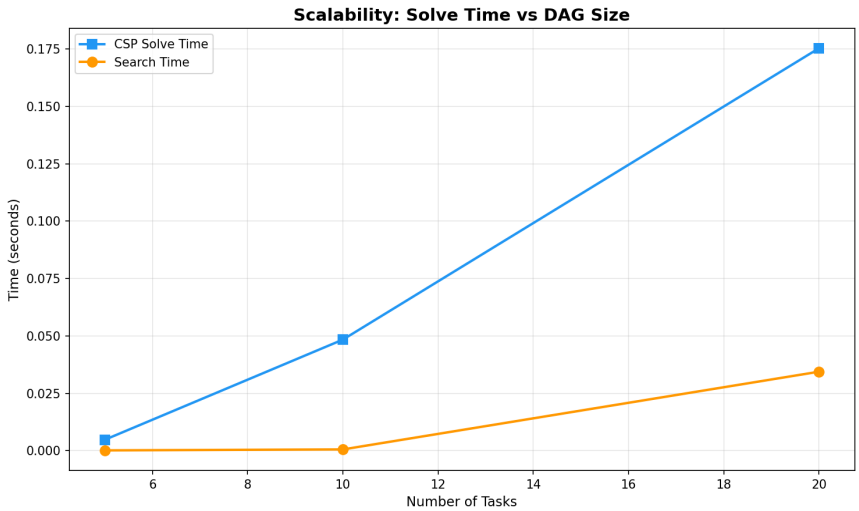

Figure 6: Scalability — solve time vs. DAG size.

4.5 Experiment 5: Failure Recovery

The medium pipeline was tested under failure rates of 0%, 10%, 20%, and 30%. At every rate, the agent detected failures, triggered the A* search planner, and produced a recovery plan using retry and skip actions. Higher failure rates reduced completed tasks as expected, but the agent consistently exercised its re-planning capability rather than allowing uncontrolled cascade failures.

Failure Rate	Completed	Failed	SLA Adherence	Re-plans
0%	10	0	100%	0
10%	2	8	100%	1
20%	2	8	100%	1
30%	1	9	100%	1

Table 6: Agent performance under varying failure rates.

4.6 Experiment 6: Agent vs. Naive Baseline

The intelligent agent was compared against a naive baseline scheduler using topological-sort FIFO assignment with no constraint reasoning, no re-planning, and no learning. In normal mode, both achieve 100% SLA adherence. The baseline shows slightly shorter makespan due

to avoiding time-slot discretization overhead. However, the baseline fundamentally lacks the agent's failure recovery (Exp 5), learning improvement (Exp 3), and constraint reasoning (Exp 1) capabilities. Figure 7 shows the comparison on the complex pipeline.

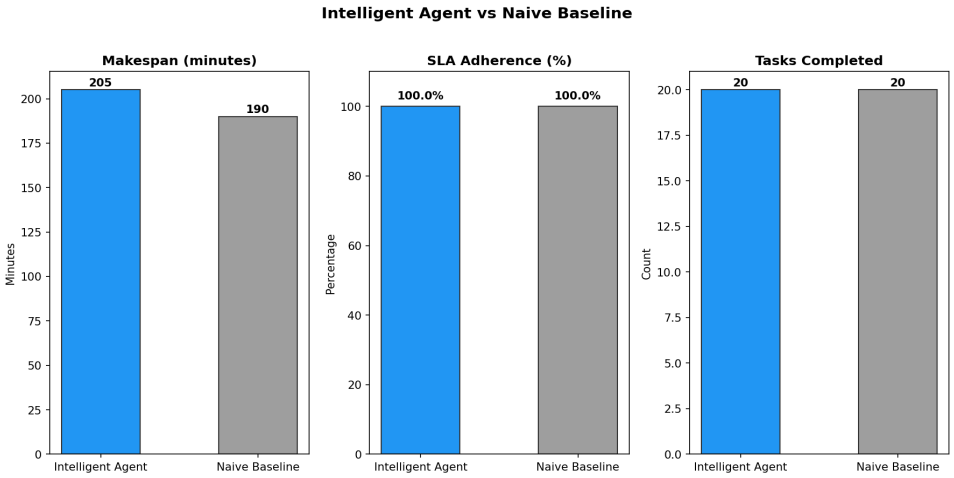

Figure 7: Agent vs. baseline comparison on the complex pipeline.

Pipeline	Agent Makespan	Baseline Makespan	Agent SLA	Baseline SLA
Simple	130m	120m	100%	100%
Medium	175m	145m	100%	100%
Complex	205m	190m	100%	100%

Table 7: Intelligent agent vs. naive baseline.

5. Discussion

5.1 Strengths and Limitations

The system demonstrates several strengths. The integration of four AI techniques creates a synergistic architecture where each component enhances the others: the KB's proactive risk detection informs CSP priority ordering, the CSP's constraint guarantees ensure schedule validity, the search planner's optimal recovery preserves integrity after disruptions, and learning improves all components over time. The CSP solver produces provably valid schedules with zero violations across all test cases. The modular design enables independent testing via 176 comprehensive automated tests.

Several limitations should be acknowledged. Testing covered pipelines up to 20 tasks; larger real-world pipelines (hundreds or thousands of tasks) may challenge the backtracking solver's exponential worst-case complexity. Time discretization (15-minute slots) introduces granularity that can cause slightly longer makespans compared to continuous-time formulations. The resource model considers only CPU and memory, omitting network bandwidth, disk I/O, and software licensing constraints that may be relevant in production environments.

5.2 Lessons Learned

Formulating scheduling as a CSP proved natural and effective, as pipeline constraints map directly to CSP variables, domains, and constraints. Forward checking was particularly valuable in pruning the search space early — experimental comparison confirmed it reduces nodes explored versus plain backtracking. EWMA learning, while algorithmically simple, achieved a 30.2% reduction in estimation error over just ten iterations, demonstrating that even lightweight learning mechanisms add meaningful value. The closed-loop architecture validated the principle that AI components become more effective when integrated through structured information sharing.

5.3 Future Improvements and Extensions

Several extensions could enhance the system for production use. Integration with Apache Airflow or Dagster would allow the agent to schedule real pipeline runs. Multi-objective optimization using Pareto frontiers could balance competing goals (minimize makespan, maximize utilization, minimize cloud cost). More sophisticated learning approaches — contextual bandits or regression models accounting for day-of-week effects, data volume trends, and resource contention — could further improve estimation accuracy. Scaling to larger pipelines would benefit from local search methods such as min-conflicts or iterative improvement algorithms. Real-time streaming integration could extend the agent to handle continuous data pipelines in addition to batch workloads.

5.4 Ethical Considerations and Societal Impact

Algorithmic scheduling raises important ethical considerations regarding transparency, fairness, and accountability. In multi-team environments, a scheduling agent that prioritizes certain teams' pipelines based on encoded priority rules could create inequitable access to shared compute resources, disadvantaging teams with lower-priority workloads. The system addresses transparency by maintaining a complete inference log of all rule firings, CSP decisions, and search actions, enabling full auditability of scheduling decisions. Future deployment should incorporate fairness constraints to ensure equitable resource allocation across organizational units. Additionally, automated scheduling reduces the need for on-call manual intervention, improving engineer quality of life by reducing 3 AM pages — a positive societal impact. However, organizations should ensure that reduced human oversight does not lead to undetected systematic errors in data pipelines that feed business-critical decisions.

6. Individual Contributions

Team Member	Primary Modules	Report Sections	Work %
Sanjana	Project setup, data model design, testing coordination, code review	Sections 1, 7, report formatting	20%
Raghava Thyagaraj	scheduler_agent.py, learning.py, main.py, system architecture	Sections 2, coordination	20%
Monisha	knowledge_base.py, inference engine, forward/backward chaining	Section 3.1, Section 5	20%
Diogo Dcosta	csp_solver.py, backtracking search, MRV/LCV/FC/AC-3	Section 3.2, Section 4	20%
Rakshitha	search_planner.py, simulator.py, visualizer.py, evaluation suite	Sections 3.3, 3.4, demo video	20%

Table 8: Individual contributions and work distribution.

All team members participated in integration testing, code review, final report review, and project planning. The project was managed using Git version control with regular commits and collaborative code reviews.

Signed by all team members:

_____ Sanjana

_____ Raghava Thyagaraj

_____ Monisha

_____ Diogo Dcosta

_____ Rakshitha

Date: _____

7. References

[1] Russell, S. and Norvig, P. (2020). *Artificial Intelligence: A Modern Approach*, 4th Edition. Pearson. Chapters 1-6, 10-11.

[2] Mackworth, A. K. (1977). "Consistency in Networks of Relations." *Artificial Intelligence*, 8(1), pp. 99-118.

[3] Hart, P. E., Nilsson, N. J., and Raphael, B. (1968). "A Formal Basis for the Heuristic Determination of Minimum Cost Paths." *IEEE Transactions on Systems Science and Cybernetics*, 4(2), pp. 100-107.

[4] Kumar, V. (1992). "Algorithms for Constraint-Satisfaction Problems: A Survey." *AI Magazine*, 13(1), pp. 32-44.

[5] Hagberg, A. A., Schult, D. A., and Swart, P. J. (2008). "Exploring Network Structure, Dynamics, and Function using NetworkX." *Proceedings of the 7th Python in Science Conference*, pp. 11-15.

[6] Hunter, J. D. (2007). "Matplotlib: A 2D Graphics Environment." *Computing in Science & Engineering*, 9(3), pp. 90-95.

[7] Pinedo, M. L. (2016). *Scheduling: Theory, Algorithms, and Systems*, 5th Edition. Springer.

[8] Apache Software Foundation. (2024). *Apache Airflow Documentation*. Retrieved from https://airflow.apache.org/docs/